

# Algoritmos com Funções e Procedimentos

---

## Tema da Aplicação

Missão no jogo onde o jogador instala sensores de presença em postes. Após a instalação, os postes acendem automaticamente quando o jogador se aproxima. O código foi implementado em C# usando a engine Unity.

### Explicação Geral do Algoritmo

A lógica é dividida em duas classes:

- PosteSensor: gerencia a instalação do sensor e a ativação da luz.
- SensorNaMao: exibe o sensor visualmente na mão do jogador durante a missão.

## A programação utiliza:

- Procedimentos (void) para ações como exibir mensagens ou ativar/desativar objetos.
- Uma função do tipo Coroutine (IEnumerator) para simular o tempo de instalação com animação.

- Controle de fluxo com estruturas if e else if.

Funções e Procedimentos Utilizados

void Update() – PosteSensor

```
private void Update()
{
    if (playerPerto && !sensorInstalado &&
MissaoPostesController.Instance.missaoIniciada &&
Input.GetKeyDown(KeyCode.E))
    {
        StartCoroutine(InstalarSensor());
    }

    if (sensorInstalado && jogador != null && luzPoste != null)
    {
        float distancia = Vector3.Distance(jogador.position,
transform.position);

        if (distancia <= distanciaParaAtivar)
        {
            luzPoste.SetActive(true);
        }
        else
        {
            luzPoste.SetActive(false);
        }
    }
}
```

---

IEnumerator InstalarSensor() – PosteSensor

```
private IEnumerator InstalarSensor()  
{  
    sensorInstalado = true;  
    playerAnimator?.SetTrigger("Interagir");  
    yield return new WaitForSeconds(tempoAnimacao);  
  
    if (sensorVisual != null) sensorVisual.SetActive(true);  
  
    MissaoPostesController.Instance.SensorInstalado();  
}
```

---

void Update() – SensorNaMao

```
void Update()  
{  
    if (MissaoPostesController.Instance.missaoIniciada &&  
!MissaoPostesController.Instance.missaoConcluida && sensorAtual  
== null)  
    {  
        sensorAtual = Instantiate(sensorPrefab, playerHand.position,  
playerHand.rotation);  
        sensorAtual.transform.SetParent(playerHand);  
    }  
    else if (MissaoPostesController.Instance.missaoConcluida &&  
sensorAtual != null)  
    {  
        Destroy(sensorAtual);  
    }  
}
```

---

## **Estrutura de Decisão: if, else if e Boas Práticas**

A estrutura if permite tomar decisões no código com base em condições. Nos códigos:

- if é usado para:

- Verificar se o jogador está apto a instalar o sensor.
- Verificar a distância do jogador para ativar/desativar a luz.
- Verificar se a missão está em andamento para exibir o sensor na mão.

- else if é usado para:

- Executar ações alternativas quando a missão já foi concluída.

### **Vantagens:**

- Organiza o código em blocos lógicos.
- Permite reações específicas a diferentes situações do jogo.
- Aumenta a legibilidade e manutenção do código.

### **Conclusão**

A missão de instalação de sensores foi implementada com o uso claro de funções e procedimentos, respeitando os princípios da lógica estruturada. O uso de if e else if demonstra decisões condicionais bem definidas, contribuindo para a clareza e funcionalidade do sistema. Essa estrutura mantém o projeto organizado e de fácil entendimento, conforme se foi ensinado em sala de aula.